



UNIVERSITATEA DIN
BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI
INFORMATICĂ



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

PLATFORMĂ BAZATĂ PE DOVEZI
PENTRU MONITORIZAREA
ANTRENAMENTELOR,
DESCOPERIREA DE PROGRAME,
COACHING ȘI CĂUTARE ÎN
ISTORICUL DE ANTRENAMENT

Absolvent

[Prenume NUME]

Coordonator științific

[Titlul și numele coordonatorului]

București, iulie 2026

Rezumat

Lucrarea descrie o aplicație web pentru antrenamente de forță, construită în jurul unei idei simple: istoricul de antrenament trebuie să rămână corect în timp. Atunci când un antrenament este efectuat, planul din care a pornit este copiat („înghețat”) sub formă de instantanee, așa că dacă utilizatorul modifică sau șterge mai târziu datele de la care a plecat, ce a fost deja înregistrat nu se mai schimbă. Aplicația permite gestionarea programelor de antrenament, executarea sesiunilor, urmărirea progresului, un marketplace de programe (publicare, vot, salvare, copiere), o analiză a structurii programului bazată pe reguli simple și explicabile (pornite din literatura de specialitate, fără pretenții medicale), un mod de coaching prin care un antrenor poate vedea datele unui client doar cu acordul lui, și o căutare full-text peste programe și peste istoric. Căutarea este un model de citire separat, construit peste OpenSearch și ținut la zi prin evenimente declanșate după ce datele se salvează în baza de date. Partea principală a lucrării este o comparație concretă între OpenSearch și PostgreSQL, făcută pe un set de date generat și crescut până la 2 milioane de documente. Backend-ul este scris în Spring Boot 3 / Java 21, cu PostgreSQL 16 ca sursă unică de adevăr, iar corectitudinea este verificată cu teste de integrare care rulează pe baze de date reale (Testcontainers, fără mock-uri). Rezultatele arată că cele două soluții se echivalează în jurul a 10.000 de documente și că cea mai mare diferență apare la căutarea care tolerează greșelile de tastare.

Abstract

This thesis describes a web application for strength training, built around one simple idea: the training history has to stay correct over time. When a workout is performed, the plan it started from is copied (“frozen”) as snapshots, so if the user later edits or deletes the data they started from, what was already recorded does not change. The application lets users manage training programs, run sessions, follow their progress, use a program marketplace (publish, vote, save, copy), get an analysis of the program structure based on simple, explainable rules (taken from the training literature, with no medical claims), use a coaching mode where a coach can see a client’s data only with that client’s consent, and search the programs and the history with full-text search. The search is a separate read model, built on top of OpenSearch and kept up to date through events fired after the data is saved to the database. The main part of the thesis is a concrete comparison between OpenSearch and PostgreSQL, run on a generated data set scaled up to 2 million documents. The backend is written in Spring Boot 3 / Java 21, with PostgreSQL 16 as the single source of truth, and correctness is checked with integration tests that run on real databases (Testcontainers, no mocks). The results show that the two options are roughly equivalent around 10,000 documents and that the biggest difference shows up on typo-tolerant (fuzzy) search.

Cuprins

1	Introducere	6
1.1	Tipul lucrării și încadrarea temei	6
1.2	Prezentarea generală a temei	6
1.3	Scopul și motivația alegerii temei	7
1.4	Contribuția proprie (pe scurt)	7
1.5	Structura lucrării	8
2	Preliminarii	9
2.1	Tehnologii de bază	9
2.2	Regăsirea informației și căutarea full-text	9
2.3	Modele de citire separate și consistență eventuală	10
2.4	Imuabilitatea istoricului	10
2.5	De unde vin regulile analizorului	11
2.6	Obiectivele lucrării	11
3	Contribuția proprie	12
3.1	Cerințe și prezentare generală a sistemului	12
3.2	Arhitectura sistemului	12
3.3	Modelul de date și principiul instantaneelor	13
3.4	Modelul de securitate	14
3.5	Funcționalitățile aplicației	15
3.6	Analizorul structural informat de dovezi	16
3.7	Modelul de citire pentru căutare (OpenSearch)	16
3.8	Evaluarea empirică: OpenSearch vs. PostgreSQL	17
3.9	Modul de coaching: RBAC plus ReBAC	19
3.10	Strategia de testare	19
3.11	Direcții de dezvoltare proiectate	19
4	Concluzii	21
4.1	Concluzii privind realizarea temei	21
4.2	Aprecieri personale privind relevanța rezultatelor	21

4.3 Dezvoltări ulterioare	22
Bibliografie	23
Anexa 1 — Capturi de ecran ale interfeței	25
Anexa 2 — Schema bazei de date	28
Anexa 3 — Listări de cod reprezentative	29
Anexa 4 — Rezultatele complete ale evaluării	31
Anexa 5 — Contractul API (selecție)	33
Anexa 6 — Configurație și rulare	34

Capitolul 1

Introducere

1.1 Tipul lucrării și încadrarea temei

Aceasta este o lucrare de licență aplicativă, din zona aplicațiilor web full-stack. Pe lângă partea clasică de inginerie software (baze de date, securitate, testare), are și o componentă care ține de regăsirea informației: o căutare full-text construită peste motorul OpenSearch. Partea cea mai interesantă din punct de vedere științific este compararea acestei căutări cu ce poate face PostgreSQL de unul singur.

1.2 Prezentarea generală a temei

Aplicațiile de monitorizare a antrenamentelor sunt, în general, de două feluri. Unele sunt doar jurnale: rețin ce ai făcut, dar nu te ajută să planifici. Altele dau recomandări de tip „antrenor cu inteligență artificială”, pe care utilizatorul nu le poate verifica și care par mai sigure decât sunt în realitate. Pe lângă asta, multe dintre ele tratează istoricul ca pe niște date care se pot schimba: dacă îți modifici sau îți ștergi un program, antrenamentele vechi pot deveni incoerente.

Platforma pe care am construit-o pleacă de la o poziție diferită. În primul rând, tratează istoricul ca pe ceva ce nu trebuie să se mai schimbe: la execuția unui antrenament, planul este copiat în instantanee, iar istoricul rămâne corect indiferent ce se întâmplă mai târziu cu datele sursă. În al doilea rând, dă un feedback pe care utilizatorul îl poate înțelege: o analiză bazată pe reguli clare, pornite din ce se știe despre antrenamentul de forță, care nu pretinde a fi sfat medical. În al treilea rând, duce căutarea dincolo de un simplu filtru SQL, printr-un model de citire făcut special pentru căutare, și măsoară concret când merită această alegere.

1.3 Scopul și motivația alegerii temei

Scopul meu a fost să construiesc un sistem cât mai apropiat de unul real, nu o aplicație de tip „temă de an”. Am vrut să arăt că pot lucra cu lucruri importante din inginerie software: securitate (autentificare cu token-uri, control al accesului pe roluri și pe relații), integritatea datelor (instantanee care nu se modifică, constrângeri puse la nivelul bazei de date), modele de citire separate, precum și o disciplină serioasă de testare.

Am ales căutarea ca temă centrală din două motive. Pe de o parte, o căutare full-text care tolerează greșelile, ordonează rezultatele după relevanță și oferă fațete chiar este utilă pentru un istoric mare de antrenamente (genul „găsește acel antrenament din 2020”). Pe de altă parte, alegerea dintre a folosi baza de date așa cum e și a adăuga un motor de căutare separat este o decizie de arhitectură luată des, dar adesea fără să fie cu adevărat justificată. Eu am transformat-o într-o întrebare la care se poate răspunde cu cifre: *de la ce dimensiune începe să merite un motor de căutare dedicat?*

1.4 Contribuția proprie (pe scurt)

Am proiectat și implementat singur întregul sistem. Mai exact:

- un model de date cu integritate puternică (instantanee care nu se modifică, ștergere logică, indecși unici parțiali, constrângeri `CHECK`), validat de Hibernate în mod `validate`;
- un sistem de securitate cu token-uri JWT și rotația token-urilor de reîmprospătare, control al accesului pe roluri (RBAC) și pe relații (ReBAC) pentru coaching, gândit să nu permită atacuri de tip IDOR (răspunde cu 404, nu cu 403);
- un analizor al structurii programelor, bazat pe reguli și ușor de explicat, cu un scor pe 100 de puncte împărțit în cinci părți și cu avertismente motivate;
- un model de citire separat pentru căutare, construit peste OpenSearch și ținut la zi prin evenimente după salvare, cu securitate pe două straturi;
- o comparație practică între OpenSearch și PostgreSQL pe un set de date generat, crescut de la 1.000 la 2.000.000 de documente, cu o variantă de PostgreSQL făcută corect (`tsvector`+GIN, `pg_trgm`);
- o suită de teste de integrare care rulează pe baze de date reale (PostgreSQL și OpenSearch prin Testcontainers, fără mock-uri și fără H2).

1.5 Structura lucrării

Lucrarea este organizată astfel. Capitolul 2 (*Preliminarii*) explică noțiunile și tehnologiile pe care se sprijină tema și unde se află astăzi acest domeniu. Capitolul 3 (*Contribuția proprie*) descrie arhitectura sistemului, modelul de date și ideea instantaneelor, securitatea, analizorul, căutarea peste OpenSearch, comparația OpenSearch vs. PostgreSQL, modul de coaching, testarea și ce am proiectat deja pentru viitor. Capitolul 4 (*Concluzii*) adună rezultatele și discută ce ar urma. Detaliile mai voluminoase — capturi din interfață, schema completă a bazei de date, bucăți de cod, tabelele cu toate măsurătorile și lista de endpoint-uri — sunt puse în anexe și sunt referite din text.

Capitolul 2

Preliminarii

În acest capitol explic, pe scurt, tehnologiile și ideile pe care se bazează lucrarea, unde se află acum domeniul și ce obiective mi-am propus pornind de la acest context.

2.1 Tehnologii de bază

Backend. Partea de server este scrisă în Java 21 cu Spring Boot 3 [14] și folosește module pentru web (REST), securitate, validare și acces la date prin Spring Data JPA / Hibernate. Datele sunt ținute în PostgreSQL 16 [11], iar schema este gestionată numai prin migrări Flyway. Un detaliu pe care l-am respectat este modul `ddl-auto: validate`: Hibernate nu creează și nu modifică niciodată schema, ci doar verifică la pornire că entitățile se potrivesc cu ea. Așa prind din start orice nepotrivire.

Frontend. Interfața este o aplicație de tip single-page scrisă în React cu TypeScript [6]. Este servită chiar de backend, de la aceeași adresă, ceea ce simplifică partea de cookie-uri și de securitate.

Autentificare. Folosesc *JSON Web Tokens* (JWT) [4]: un token de acces de scurtă durată, semnat, trimis în antetul `Authorization`, și un token de reîmprospătare opac, din care în baza de date păstrez doar o amprentă. Parolele sunt stocate ca amprente generate cu o funcție gândită să reziste la atacuri prin dicționar.

2.2 Regăsirea informației și căutarea full-text

Regăsirea informației (information retrieval) se ocupă de cum găsești, dintr-o mulțime de documente, pe cele potrivite pentru o căutare. Spre deosebire de o potrivire exactă în SQL, căutarea full-text presupune mai multe lucruri: împărțirea textului în cuvinte și normalizarea lui (inclusiv scoaterea diacriticelor), un *index inversat* care leagă fiecare

cuvânt de documentele în care apare, o funcție care ordonează rezultatele după cât de potrivite sunt și un mod de a tolera greșelile.

Relevanță și ordonare. Motoarele moderne folosesc funcția de scor *BM25* [9], o variantă mai bună din familia TF-IDF, care ține cont și de cât de des apare un cuvânt, și de cât de lung este documentul. Cât de bună este ordonarea se măsoară cu indicatori ca *precision@k* și *nDCG* [3], care răsplătesc rezultatele relevante când apar mai sus în listă.

Toleranța la greșeli. Căutarea „fuzzy” acceptă potriviri aproximative, pe baza distanței de editare (Levenshtein). Este utilă pentru greșelile de tastare (de exemplu *benhc* → *bench*). În bazele de date relaționale, un lucru asemănător se obține cu extensia *pg_trgm*, care compară cuvintele după trigrame.

Căutarea în PostgreSQL vs. motoare dedicate. PostgreSQL știe să facă singur căutare full-text, prin tipul *tsvector*, funcții de ordonare ca *ts_rank_cd* și indecși GIN [12]. La dimensiuni mici este suficient și are avantajul că totul stă într-un singur loc. Motoarele dedicate, cum este OpenSearch [7] (o variantă open-source a Elasticsearch, construită peste biblioteca Lucene), oferă în schimb analizoare configurabile, sinonime, scoruri BM25, fațete prin agregări și o căutare cu greșeli care rămâne rapidă chiar și la volume mari. Care variantă e mai bună depinde de scară, iar asta am verificat experimental.

2.3 Modele de citire separate și consistență eventuală

Ca să separ partea de scriere (tranzacțională, normalizată) de partea de citire (optimizată pentru interogări), am folosit un *model de citire separat*, construit dintr-o singură sursă de adevăr. Este o formă mai relaxată a tiparului CQRS [5]. Modelul separat (la mine, indexul OpenSearch) este *eventual consistent* cu sursa de adevăr (PostgreSQL) și poate fi reconstruit oricând. Partea grea aici este să duci modificările în mod sigur către modelul separat (problema *dual-write*), pe care o discut în Capitolul 3.

2.4 Imuabilitatea istoricului

Ideea că datele istorice ar trebui tratate ca fapte care nu se mai rescriu este un principiu cunoscut în sistemele care lucrează cu multe date [2]. La mine, un antrenament efectuat este un fapt: nu trebuie să se schimbe doar pentru că am editat ulterior planul din care a pornit. Am pus asta în practică prin *instantanee* (snapshots) și chei externe care pot fi anulate, detaliate în Secțiunea 3.3.

2.5 De unde vin regulile analizorului

Analiza structurii unui program pleacă de la rezultate din literatura antrenamentului de forță, mai ales de la legătura dintre volumul săptămânal (numărul de serii pe grupă musculară) și creșterea musculară [10]. Folosesc aceste rezultate ca *reguli* pentru a semnala probleme de structură (de exemplu volum prea mic sau prea mare, sau lipsa unui tipar de mișcare), nu ca rețete individuale. De aceea analizorul este determinist, ușor de explicat și prudent, iar la fiecare rezultat adaugă un avertisment despre limitele lui.

2.6 Obiectivele lucrării

Pornind de aici, obiectivele mele au fost:

1. să construiesc o platformă full-stack funcțională, cu date bine protejate și o securitate serioasă;
2. să fac un model de citire pentru căutare, separat și eventual consistent, a cărui securitate nu poate fi ocolită din parametrii cererii;
3. să verific concret, pe un set de date care crește, de la ce dimensiune merită un motor de căutare dedicat față de ce poate face PostgreSQL singur;
4. să demonstrez corectitudinea prin teste de integrare pe infrastructură reală, nu pe substitute.

Capitolul 3

Contribuția proprie

3.1 Cerințe și prezentare generală a sistemului

Pe scurt, ce poate face un utilizator autentificat: își gestionează datele de planificare (un catalog de exerciții oficiale, comun, plus exerciții proprii, rutine de încălzire/încheiere, săli și echipamente), își construiește *programe* de antrenament pe mai multe zile, execută sesiuni live (unde notează seriile cu greutate, repetări, RPE), își vede istoricul și progresul, publică și copiază programe într-un *marketplace*, primește o analiză a structurii unui program și, dacă este antrenor, poate vedea (doar citi) datele unui client cu care are o relație activă. Peste programe și peste istoric funcționează o *căutare* full-text.

Cerințele nefuncționale au ghidat tot ce am făcut: PostgreSQL este sursa unică de adevăr; schema este fixată de la început și se schimbă doar prin migrări Flyway; fiecare resursă a unui utilizator verifică proprietarul, ca să nu existe IDOR; iar codul este gândit la o calitate de tip „portofoliu / sisteme back-end”, fără substitute în memorie, fără H2, cu teste de integrare reale. Câteva capturi din interfață se găsesc în **Anexa 1**.

3.2 Arhitectura sistemului

Sistemul este o aplicație web pe trei niveluri: o aplicație React (single-page) vorbește prin JSON cu un API REST scris în Spring Boot, care salvează în PostgreSQL. În producție, frontend-ul compilat este inclus în backend și servit de la aceeași adresă (<http://localhost:8080/>), așa că tot cookie-ul de autentificare rămâne same-origin. Tot stack-ul rulează prin Docker Compose (**Anexa 6**).

Backend-ul este organizat *pe funcționalități*, nu pe niveluri tehnice. Fiecare pachet (auth, exercise, routine, gym, template, session, history, analytics, marketplace, analyzer, search, coaching, shared) își are propriile patru niveluri: domain (entitățile JPA), application (servicii, comenzi, excepții), infrastructure (depozite Spring Data și SQL nativ) și web (controllere și DTO-uri). O regulă pe care am ținut-o strict: nivelul

web nu expune niciodată entități, ci doar DTO-uri. Când o funcționalitate are nevoie de datele alteia (de exemplu analizorul citește structura unui program), cel care le deține expune un *model de citire la nivel de aplicație*, nu un DTO web și nici un depozit; așa dependențele rămân curate.

O cerere este autentificată de un filtru care verifică JWT-ul și pune în contextul de securitate un `UserPrincipal`. Controllerele citesc acest principal și nu se încred niciodată într-un identificador de utilizator venit din corpul cererii. Erorile de domeniu extind o excepție comună care duce cu ea un cod HTTP și un cod de eroare stabil, ușor de citit de un program, iar un singur handler le transformă într-un format de eroare uniform (**Anexa 5**). Diagrama de arhitectură este în Figura 3.1.

*[Substituent imagine] Diagrama de arhitectură: SPA React → API
REST Spring Boot → PostgreSQL (sursa de adevăr) și OpenSearch
(model de citire separat).
(se va înlocui cu `\includegraphics` după adăugarea capturii în `images/`)*

Figura 3.1: Arhitectura de ansamblu a sistemului.

3.3 Modelul de date și principiul instantaneelor

Schema este creată de o singură migrare inițială (V1), care acoperă utilizatorii, catalogul de exerciții și grupele musculare, rutinele, sălile și echipamentele, programele cu tot ce conțin (zile / exerciții / rutine), sesiunile live cu instantaneele lor, tabelele de marketplace (voturi, salvări, evenimente de utilizare, statistici per program) și câteva tabele lăsate pentru mai târziu (coaching, rezultate de analiză, un *outbox*). A doua migrare (V2) pune catalogul oficial de exerciții, iar a treia (V3) adaugă un index pentru concurență. Toate tabelele și constrângerile sunt în **Anexa 2**.

Principiul instantaneelor. Faptul că istoricul rămâne corect este cerința mea de bază și o asigur în două locuri. Întâi, exercițiile dintr-o zi de program rețin `exercise_name_snapshot`, `exercise_type_snapshot` și o copie a grupelor musculare, păstrând doar o legătură *care poate fi anulată* către exercițiul viu (`ON DELETE SET NULL`). Apoi, când pornește un antrenament, fac un al doilea instantaneu: numele programului, al zilei și al sălii, plus numele/tipul/valorile planificate ale fiecărui exercițiu și grupele musculare, sunt copiate în `session_exercises`; seriile rețin și numele echipamentului. Pentru că rândurile sesiunii își poartă propriile valori și au doar chei externe care pot fi anulate,

dacă editez sau șterg programul, exercițiul, rutina, sala sau echipamentul, ce a fost notat într-o sesiune veche nu se schimbă. Asta o verific direct prin teste.

Ștergere logică, unicitate și constrângeri. Resursele de planificare folosesc o coloană `deleted_at`, ca istoricul care le referă să supraviețuiască. Unicitatea o impun cu *indecși unici parțiali* care ignoră rândurile șterse (de exemplu, un nume unic de exercițiu propriu per utilizator `WHERE deleted_at IS NULL`), așa că un nume poate fi reutilizat după ștergere. Multe reguli sunt impuse chiar de baza de date prin constrângeri `CHECK`: valori permise pentru enumerări, `days_per_week BETWEEN 1 AND 7`, `rpe BETWEEN 1.0 AND 10.0`, `finished_at >= started_at`, contoare care nu pot fi negative, interdicția de a te antrena singur (`coach_user_id <> client_user_id`) și regula că un program public trebuie să aibă o dată de publicare.

Concurență. Regula „un singur antrenament activ per utilizator” este garantată de un *index unic parțial* `ux_workout_sessions_one_active_per_user ... WHERE status = 'IN_PROGRESS'` (migrarea V3). Verificarea din serviciu este doar o cale rapidă și prietenoasă; dacă două cereri pornesc în același timp, cea care pierde primește un `409 ACTIVE_WORKOUT_EXISTS` curat, nu o eroare 500. Contoarele de marketplace (voturi, salvări) sunt actualizate cu SQL atomic și mărginit (`GREATEST(0, contor +/- delta)`), deci nu apare cursa citire-modificare-scriere și nu ajung niciodată sub zero.

3.4 Modelul de securitate

Parolele sunt stocate ca amprente BCrypt. La autentificare, API-ul dă un token de acces JWT de scurtă durată (HS256), pe care SPA-ul îl trimite în antetul `Authorization` și îl ține doar în memorie, și un token de reîmprospătare opac, ca un cookie `HttpOnly, SameSite=Strict`, limitat la calea `/api/auth`. Din token-ul de reîmprospătare păstrez în baza de date doar o amprentă SHA-256. La reîmprospătare, token-ul este *rotit*: cel prezentat este verificat și revocat, iar o pereche nouă este emisă, totul într-o singură tranzacție. Așa, dacă cineva fură un token și încearcă să-l refolosească, este prins (amprenta lui există, dar are `revoked_at` setat). Am dezactivat protecția CSRF intenționat, fiindcă token-ul de acces vine din antet, nu din cookie; singurul cookie cu credențiale este cel de reîmprospătare, protejat de attributele de mai sus.

Autorizare și siguranță IDOR. Fiecare resursă a unui utilizator este încărcată cu o interogare care include și identificatorul proprietarului. Important, dacă cineva cere o resursă a altui utilizator, primește **404**, nu **403**, ca să nu afle nici măcar că resursa există. Resursele imbricate își verifică proprietarul în lanț (de exemplu, un exercițiu dintr-o zi de program este accesibil doar dacă programul acelei zile îți aparține). În configurația de

securitate, ordinea este: rute publice de autentificare; `/api/admin/**` doar pentru rolul ADMIN; `/api/coach/**` doar pentru rolul COACH; restul `/api/**` doar pentru utilizatori autentificați.

3.5 Funcționalitățile aplicației

Date de planificare și programe. Utilizatorii își fac o bibliotecă de exerciții (catalogul oficial plus exerciții proprii, fiecare cu grupe musculare primare/secundare), rutine, săli și echipamente. Un program este privat și pe mai multe zile; fiecare zi are exerciții ordonate (cu serii/repetări/greutate/pauză planificate) și rutine atașate. După fiecare modificare a exercițiilor, recalculez pe partea de PostgreSQL niște vectori agregați ai programului (grupe musculare, identificatori de exerciții oficiale, nume de exerciții), printr-o singură instrucțiune nativă cu `array_agg(DISTINCT ... ORDER BY ...)`, ca să pregătesc căutarea.

Execuția live și istoricul. Pornirea unui antrenament face, într-o singură tranzacție, copierea-instantaneu din Secțiunea 3.3. În timpul sesiunii, utilizatorul notează serii, adaugă exerciții în plus, neplanificate, și la final finalizează sau anulează sesiunea. Istoricul listează sesiunile cele mai recente primele, cu câteva numere de sumar per sesiune calculate printr-o singură interogare în lot (ca să evit $N+1$).

Analize de progres. Analizele sunt calculate complet în PostgreSQL, peste sesiunile finalizate: volumul total pe zi, numărul de antrenamente pe săptămână (`date_trunc`), distribuția seriilor pe grupa musculară primară și, pentru un grafic al progresului la forță, cea mai bună estimare a unei repetări maxime (1RM) pe exercițiu pe zi, cu formula lui Epley [1]:

$$1RM \approx w \cdot \left(1 + \frac{r}{30}\right), \quad (3.1)$$

unde w este greutatea și r numărul de repetări ale seriei.

Marketplace. Proprietarii publică un program nevid, făcându-l public. Utilizatorii autentificați parcurg programele publice (sortate după cele mai noi, scor sau tendință), votează, salvează și *folosesc* un program. „Folosirea” copiază în profunzime programul public într-un program nou, privat, al celui care copiază. Legătura către exercițiu o păstrez doar dacă este un exercițiu *oficial*; altfel o anulez, dar păstrez copia numelui/tipului/grupelor musculare — așa copia rămâne complet utilizabilă și independentă de original, fără să scape date private.

3.6 Analizorul structural informat de dovezi

Analizorul este determinist și bazat pe reguli — nu folosește învățare automată. Dă un *Scor de structură a programului* (nu o judecată de „calitate științifică” sau medicală), iar fiecare răspuns vine cu un avertisment că nu este sfat medical și că nu ține cont de recuperare, tehnică, efort, somn, alimentație sau de cum răspunde fiecare individ. Îl calculez la cerere, la un GET read-only (`/api/templates/{id}/analysis`), și îl folosesc și la indexarea pentru căutare.

Volumul săptămânal este suma peste zilele scrise în program, iar seriile sunt ponderate după rolul muscular (primar 1.0, secundar 0.5). Exercițiile fără serii planificate sunt *scoase* din volum (nu le presupun o valoare implicită) și marcate ca date incomplete; doar exercițiile de forță contează la volum și echilibru. Scorul de 100 de puncte este împărțit în cinci părți, fiecare pornind de la maximul ei și scăzând o valoare fixă pentru fiecare problemă găsită (dar nu sub zero), deci rezultatele sunt complet deterministe și pot fi testate (Tabelul 3.1). Categoriile pun totalul în WELL_STRUCTURED (≥ 80), DECENT_STRUCTURE (55–79) sau NEEDS_REVIEW (< 55). Familiile de reguli și o bucată reprezentativă din logica de scor sunt în **Anexa 3**.

Sub-scor	Maxim	Exemple de penalizări
Volum și acoperire	35	regiune absentă –12; volum excesiv –5; foarte scăzut –3
Frecvență	20	volum concentrat într-o singură zi –4
Echilibru	20	dezechilibru sever –8; moderat –4
Designul sesiunii	15	sesiune excesivă –5; concentrare pe o grupă –3
Specificitate / pauză	10	pauză prea scurtă –2; interval de repetări extrem –2 ... –3

Tabela 3.1: Descompunerea scorului de structură pe cinci sub-scoruri deterministe.

3.7 Modelul de citire pentru căutare (OpenSearch)

Căutarea este un *model de citire separat*, construit peste OpenSearch și pornit printr-un indicator de configurare (`app.search.enabled`, implicit oprit, ca să nu lege compilarea locală și testele pe PostgreSQL de OpenSearch). Am două indexuri, `templates` și `workout_sessions`, cu nume versionate ascunse în spatele unor alias-uri stabile. Analizorul de indexare normalizează textul (lowercase + scoaterea diacriticelor), iar cel de căutare adaugă și un set de *sinonime* (de exemplu *pecs*→*chest*, *ohp*→*overhead press*). Interogarea de tip `multi_match` dă ponderi (boost) câmpurilor — pentru programe `name^4`, `exerciseNames^3`, `muscleGroupsText^2` etc. — și toleranță la greșeli (`fuzziness=AUTO`, cu o regulă care nu aplică „fuzzy” pentru căutări mai scurte de 3 caractere). Pentru marketplace, scorul de relevanță este combinat cu un *function_score* care adaugă o influență logaritmică a popularității (salvări, utilizări). Rezultatele includ fațete (agregări `terms` și un histogram pe lună) și evidențierea potrivirilor cu `<mark>`.

Cum scriu în index și cum țin totul consistent. Indexarea este pornită de evenimente declanșate *după* ce tranzacția se salvează (`@TransactionalEventListener(AFTER_COMMIT)`), ca să nu ajungă în index munca anulată prin rollback. Am două căi de actualizare: o reindexare *structurală* reconstruiește tot documentul și recalculează câmpurile din analiză, iar o actualizare *doar a contoarelor* (vot/salvare/utilizare) nu mai recalculează analiza. Toate scrierile sunt *idempotente* (rescriu documentul întreg după identificator). Dacă o scriere dă greș, o jurnalizez și merg mai departe — PostgreSQL rămâne sursa de adevăr, iar o reconstrucție repară indexul. Intenționat, aceasta *nu* este încă un outbox tranzacțional (vezi Secțiunea 3.11).

Securitate pe două straturi. Întâi, un *filtru obligatoriu*, luat din JWT, pe care parametrii cererii nu îl pot suprascrie: `marketplace ⇒ visibility=PUBLIC`; `my ⇒ ownerId=apelant`; `workouts ⇒ ownerId=apelant`. Apoi, o verificare suplimentară în PostgreSQL: identificatorii întorși de OpenSearch sunt verificați din nou în baza de date cu aceeași regulă, ca să arunc orice rezultat învechit (de exemplu, abia depublicat sau șters) și să adaug starea curentă de vot/salvare pentru marketplace.

Reconstrucție și reziliență. Serviciul de reconstrucție creează un index nou cu marcaj de timp, îl umple în lot din PostgreSQL și *schimbă atomic alias-ul* (fără să se oprească citirile), apoi șterge indexul vechi; dacă ceva pică, indexul nou este aruncat, iar cel vechi continuă să servească. Reconstrucția manuală (`POST /api/admin/search/reindex`) este permisă doar rolului ADMIN. Dacă căutarea este oprită sau OpenSearch nu răspunde, endpoint-urile întorc 503 și interfața trece înapoi pe lista din SQL; un index încă neconstruit este tratat ca „fără rezultate”, nu ca o eroare.

Căutarea făcută de antrenor. Un antrenor poate căuta full-text în istoricul unui client (`/api/coach/clients/{id}/search/workouts`). Este singurul loc unde căutarea rulează cu un identificator de proprietar diferit de cel care cere, și este sigură fiindcă trece mai întâi prin poarta de acces a antrenorului (vezi Secțiunea 3.9) și apoi folosește exact același serviciu de căutare, dar cu identificatorul clientului, păstrând filtrul obligatoriu și verificarea din PostgreSQL.

3.8 Evaluarea empirică: OpenSearch vs. PostgreSQL

Partea cea mai importantă din punct de vedere științific este o măsurătoare concretă: de la ce dimensiune merită un motor de căutare dedicat față de ce poate face PostgreSQL singur.

Metodologie. Varianta de PostgreSQL este făcută *corect*, nu ca să piardă: căutare full-text ponderată cu `tsvector` (`setweight` A/B/C/D) și index GIN, ordonată cu `ts_rank_cd`; căutare fuzzy cu `pg_trgm` și index GIN; fațete prin `GROUP BY`. Partea OpenSearch folosește exact aceleași mapări și interogări ca în producție. Setul de date este generat determinist și *în flux* (pe loturi, cu memorie constantă), cu un vocabular realist de ridicări, popularitate înclinată și 90%/10% public/privat. Am măsurat la șapte dimensiuni, de la 1.000 la 2.000.000 de documente, pe cinci scenarii: două căutări full-text, o căutare fuzzy (greșeală de tastare), o căutare filtrată și o agregare pe fațete. Măsurarea este pe un singur fir, cu încălzire și un buget de timp, și raportează percentilele $p_{50}/p_{95}/p_{99}$.

Rezultate. Tabelul 3.2 arată latențele la cea mai mare dimensiune (2M de documente); tabelele complete, pe toate dimensiunile, sunt în **Anexa 4**. Pe scurt: cele două soluții se echivalează în jurul a *10.000 de documente* (sub această valoare PostgreSQL e mai rapid; peste ea, OpenSearch câștigă clar); căutarea full-text în PostgreSQL crește aproape liniar (de la $\sim 1,5$ ms la $\sim 1,86$ s între 1k și 2M), în timp ce OpenSearch rămâne aproape constant (~ 3 –64 ms); *căutarea fuzzy face diferența* — `pg_trgm` se duce de la 14 ms la $\sim 15,8$ s (1k $\rightarrow$ 2M), față de ~ 4 ms, aproape constant, în OpenSearch (un raport de ordinul a $4000\times$ la 2M); în plus, indexul OpenSearch ocupă de $\sim 3,6\times$ mai puțin pe disc și se construiește de $\sim 2\times$ mai repede la 2M.

Scenariu (la 2M documente)	PostgreSQL	OpenSearch
Full-text „bench press” (p_{50})	1864 ms	64 ms
Full-text „squat” (p_{50})	1437 ms	4 ms
Fuzzy „benhc” (greșeală) (p_{50})	15825 ms	4 ms
Filtrat (press + INTERMEDIATE + PPL) (p_{50})	2295 ms	7 ms
Fațetă după dificultate (p_{50})	1193 ms	2 ms
Timp de construcție a indexului	239 s	115 s
Dimensiune pe disc	2632 MB	739 MB

Tabela 3.2: Comparatie OpenSearch vs. PostgreSQL la 2.000.000 de documente. Tabelele complete pe toate cele șapte dimensiuni sunt în Anexa 4.

Interpretare. Rezultatul transformă o alegere de arhitectură luată des „din reflex” într-o decizie cu temei: sub pragul de echivalență, complexitatea în plus a unui motor dedicat nu se justifică; peste el — și mai ales dacă ai nevoie de toleranță la greșeli — un motor dedicat devine nu doar util, ci necesar.

3.9 Modul de coaching: RBAC plus ReBAC

Modul de coaching adaugă un al doilea strat de control peste RBAC. O relație antrenor–client are un ciclu de viață (`PENDING` → `ACTIVE` dacă se acceptă, sau `REJECTED`; o relație `ACTIVE` poate fi `REVOKED` de oricare parte), cu un index unic parțial care permite o singură relație vie per pereche. RBAC (`/api/coach/**` cere rolul `COACH`) este necesar, dar nu suficient: fiecare citire per client trece printr-o poartă bazată pe relații (ReBAC) care cere o relație *activă* și, altfel, întoarce **404 (nu 403)**, ca un antrenor să nu poată afla ce utilizatori există. Accesul antrenorului este doar de citire (istoric, analize, o sesiune, căutare); nu există niciun endpoint prin care antrenorul să modifice datele clientului, iar clientul poate retrage accesul oricând.

3.10 Strategia de testare

Corectitudinea o verific cu teste de integrare pe *infrastructură reală*: o clasă de bază pornește un container PostgreSQL (Testcontainers), iar suita de căutare adaugă un container OpenSearch real și pornește indicatorul de căutare — fără mock-uri, fără H2. Așa, toată schema gestionată de Flyway și constrângerile ei sunt puse la treabă. Testele verifică comportamentul prin nivelul HTTP și, unde contează, garanțiile la nivel de bază de date: o a doua inserare „în desfășurare” brută este respinsă de indexul unic parțial; vectorii agregați sunt citați înapoi cu `JdbcTemplate`; instantaneele unei sesiuni rămân neschimbate după ce editez datele sursă. La momentul scrierii, suita are 128 de teste care trec.

3.11 Direcții de dezvoltare proiectate

Lucrurile de mai jos sunt *gândite și pregătite*, dar încă neimplementate. Le includ aici fiindcă arhitectura și, în mai multe cazuri, chiar schema bazei de date le anticipează: schema V1 conține deja, nefolosite de cod, tabelele `outbox_events`, `template_analysis_results`, `coach_session_comments` și `coach_template_assignments`.

- **Cache cu Redis pentru marketplace (cache-aside).** Aș memora paginile de listă și detaliile de program, cu invalidare condusă de aceleași evenimente după salvare deja existente, plus protecție împotriva efectului de „turmă” prin expirare probabilistică timpurie (XFetch [13]).
- **Outbox tranzacțional pentru indexare.** Ar rezolva problema *dual-write*: un rând de eveniment scris în aceeași tranzacție cu modificarea, în tabela `outbox_events` deja existentă, plus un proces care îl trimite mai departe cu livrare cel-

puțin-o-dată și un consumator idempotent (indexatorul este deja idempotent). Aș discuta și alternativa CDC/Debezium [8].

- **Evaluarea calității relevanței.** Aș duce evaluarea de la viteză la calitate: un set de judecăți etichetate și indicatori ca precision@k și nDCG [3], plus un studiu de ablație (sinonime / fuzzy / ponderi pornite sau oprite).
- **Paginare prin chei (keyset).** Aș înlocui paginarea `OFFSET` de acum cu paginare prin cheie de continuare, ducând costul paginilor adânci de la $\mathcal{O}(n)$ la $\mathcal{O}(\log n)$.
- **Limitarea ratei la autentificare** (token-bucket cu Redis) și **trecerea la Argon2id** pentru parole, cu re-hashing transparent la autentificare.
- **Observabilitate** (trasare distribuită OpenTelemetry + metrice Prometheus/Grafana), **partiționarea pe timp** a tabelii `workout_sessions`, o **coadă de lucru** bazată pe `SELECT ... FOR UPDATE SKIP LOCKED` (potrivită pentru trimiterea din outbox) și **vederi materializate** pentru analize.

Aceste direcții sunt descrise, ca proiectare, în Capitolul 4.

Capitolul 4

Concluzii

4.1 Concluzii privind realizarea temei

Am construit o platformă coerentă pentru antrenamente de forță, gândită în jurul integrității datelor: planurile devin istoric care nu se mai schimbă, prin instantanee; analizele și feedback-ul ies din date reale; iar un marketplace permite ca programele să fie partajate și copiate independent. M-am concentrat pe corectitudine (teste pe baze de date reale), pe integritatea datelor (instantanee pe două niveluri, ștergere logică, constrângeri în baza de date), pe securitate (autentificare JWT cu rotație și acces sigur față de IDOR, RBAC plus ReBAC pentru coaching) și pe explicabilitate (un analizor determinist, cu avertismente), nu pe funcționalități de fațadă.

Căutarea am tratat-o ca pe o problemă de inginerie a sistemelor: un model de citire separat, eventual consistent și reconstruibil, cu o securitate care nu poate fi ocolită din parametrii cererii. Cea mai importantă contribuție din punct de vedere științific este *comparația concretă* dintre OpenSearch și PostgreSQL, făcută pe un set de date crescut până la două milioane de documente. Ea arată că cele două soluții se echivalează în jurul a 10.000 de documente și că diferența mare apare la căutarea cu greșeli de tastare, unde varianta din baza de date ajunge la mai multe secunde, în timp ce motorul dedicat rămâne aproape constant. Astfel, o decizie de arhitectură luată de obicei „din reflex” devine una cu temei și cu cifre.

4.2 Aprecieri personale privind relevanța rezultatelor

Cred că valoarea lucrării nu stă în numărul de funcționalități, ci în felul în care au fost făcute: o singură sursă de adevăr verificată strict, o suită de teste de integrare pe infrastructură reală și o măsurătoare clară a unei alegeri de arhitectură. Aceste rezultate se pot folosi și în altă parte: ideea instantaneelor pentru păstrarea istoricului, modelul de citire separat și eventual consistent și metoda de comparare a soluțiilor de căutare se aplică

în multe alte domenii (comerț electronic, sisteme de documente, jurnale de activitate).

4.3 Dezvoltări ulterioare

Arhitectura a fost gândită ca să poată fi extinsă, iar schema bazei de date pregătește deja mai multe direcții (tabelele `outbox_events`, `template_analysis_results`, `coach_session_comments` și `coach_template_assignments` există din prima migrare, dar încă nu sunt folosite de cod). Ce aș face în continuare, descris ca proiectare în Secțiunea 3.11: (1) un strat de cache Redis de tip cache-aside pentru marketplace, cu invalidare condusă de evenimente și protecție împotriva efectului de turmă; (2) un *outbox tranzacțional* pentru indexare, care rezolvă problema dual-write folosind tabela și indexatorul idempotent deja existente; (3) o evaluare a calității relevanței (precision@k , nDCG), care completează măsurarea de viteză deja făcută; (4) paginare prin chei pentru paginile adânci; (5) limitarea ratei la autentificare și trecerea la Argon2id; și (6) observabilitate prin trasare și metrice, partiționarea pe timp a istoricului, o coadă de lucru bazată pe `SKIP LOCKED` și vederi materializate pentru analize. Fiecare dintre acestea extinde, nu schimbă, ce am construit până acum.

Bibliografie

- [1] Boyd Epley, *Poundage Chart. Boyd Epley Workout*, Lincoln, NE: Body Enterprises, 1985.
- [2] Pat Helland, „Immutability Changes Everything”, în *Communications of the ACM* 59.1 (2016), pp. 64–70, DOI: [10.1145/2844112](https://doi.org/10.1145/2844112).
- [3] Kalervo Järvelin și Jaana Kekäläinen, „Cumulated Gain-based Evaluation of IR Techniques”, în *ACM Transactions on Information Systems* 20.4 (2002), pp. 422–446, DOI: [10.1145/582415.582418](https://doi.org/10.1145/582415.582418).
- [4] Michael B. Jones, John Bradley și Nat Sakimura, *JSON Web Token (JWT)*, RFC 7519, Internet Engineering Task Force (IETF), 2015, URL: <https://www.rfc-editor.org/rfc/rfc7519> (accesat în 16.06.2026).
- [5] Martin Kleppmann, *Designing Data-Intensive Applications*, O'Reilly Media, 2017, ISBN: 978-1449373320.
- [6] Meta Open Source, *React Documentation*, 2024, URL: <https://react.dev/> (accesat în 16.06.2026).
- [7] OpenSearch Project, *OpenSearch Documentation*, 2024, URL: <https://opensearch.org/docs/latest/> (accesat în 16.06.2026).
- [8] Chris Richardson, *Microservices Patterns: With Examples in Java*, Manning Publications, 2018, ISBN: 978-1617294549.
- [9] Stephen Robertson și Hugo Zaragoza, „The Probabilistic Relevance Framework: BM25 and Beyond”, în *Foundations and Trends in Information Retrieval* 3.4 (2009), pp. 333–389, DOI: [10.1561/15000000019](https://doi.org/10.1561/15000000019).
- [10] Brad J. Schoenfeld, Dan Ogborn și James W. Krieger, „Dose-response Relationship Between Weekly Resistance Training Volume and Increases in Muscle Mass: A Systematic Review and Meta-analysis”, în *Journal of Sports Sciences* 35.11 (2017), pp. 1073–1082, DOI: [10.1080/02640414.2016.1210197](https://doi.org/10.1080/02640414.2016.1210197).
- [11] The PostgreSQL Global Development Group, *PostgreSQL 16 Documentation*, 2024, URL: <https://www.postgresql.org/docs/16/> (accesat în 16.06.2026).

- [12] The PostgreSQL Global Development Group, *PostgreSQL Documentation: Full Text Search*, 2024, URL: <https://www.postgresql.org/docs/16/textsearch.html> (accesat în 16.06.2026).
- [13] Andrea Vattani, Flavio Chierichetti și Keegan Lowenstein, „Optimal Probabilistic Cache Stampede Prevention”, în *Proceedings of the VLDB Endowment* 8.8 (2015), pp. 886–897, DOI: [10.14778/2757807.2757813](https://doi.org/10.14778/2757807.2757813).
- [14] VMware Tanzu, *Spring Boot Reference Documentation*, 2024, URL: <https://docs.spring.io/spring-boot/index.html> (accesat în 16.06.2026).

Anexa 1 — Capturi de ecran ale interfeței

Aici sunt câteva capturi din aplicație. Casetele de mai jos sunt doar substituenți; se vor înlocui cu imaginile reale (`\includegraphics`) după ce le adaug în `images/`.

*[Substituent imagine] Tabloul de bord (dashboard) după autentificare: un rezumat al activității și legăturile către funcționalități.
(se va înlocui cu `\includegraphics` după adăugarea capturii în `images/`)*

Figura 4.1: Tabloul de bord al utilizatorului.

*[Substituent imagine] Constructorul de programe: zile, exerciții ordonate cu serii/repetări/pauză planificate și rutine atașate.
(se va înlocui cu `\includegraphics` după adăugarea capturii în `images/`)*

Figura 4.2: Constructorul de programe de antrenament.

*[Substituent imagine] Sesiunea live: notarea seriilor (greutate, repetări, RPE) și adăugarea de exerciții în plus.
(se va înlocui cu \includegraphics după adăugarea capturii în images/)*

Figura 4.3: Execuția unei sesiuni de antrenament.

*[Substituent imagine] Analize de progres: volum în timp, antrenamente pe săptămână, distribuția pe grupe musculare și progresia 1RM estimat (grafice Recharts).
(se va înlocui cu \includegraphics după adăugarea capturii în images/)*

Figura 4.4: Pagina de analize și progres.

*[Substituent imagine] Marketplace cu căutare: câmp de căutare, fațete (dificultate, split, grupe musculare) și evidențierea potrivirilor (<mark>).
(se va înlocui cu \includegraphics după adăugarea capturii în images/)*

Figura 4.5: Marketplace-ul de programe cu căutare full-text și fațete.

*[Substituent imagine] Modul de coaching: căutarea full-text în istoricul unui client, accesibilă doar dacă există o relație activă.
(se va înlocui cu \includegraphics după adăugarea capturii în images/)*

Figura 4.6: Căutarea în istoricul clientului (mod coaching).

*[Substituent imagine] Panoul analizorului: scorul pe 100 de puncte,
sub-scorurile și avertismentele, fiecare cu o sugestie.
(se va înlocui cu `\includegraphics` după adăugarea capturii în `images/`)*

Figura 4.7: Rezultatul analizorului structural de programe.

Anexa 2 — Schema bazei de date

Toată schema este creată de migrarea Flyway V1 (28 de tabele), umplută cu date oficiale de V2 și completată de V3 cu un index pentru concurență. Tabelele, grupate pe funcționalități, sunt:

- **Autentificare / coaching:** `app_users`, `refresh_tokens`, `coach_profiles`, `coach_client_relationships`.
- **Catalog / planificare:** `muscle_groups`, `exercises`, `exercise_muscle_groups`, `routines`, `gyms`, `equipment`.
- **Programe:** `workout_templates`, `template_stats`, `template_days`, `template_day_exercises`, `template_day_exercise_muscle_groups`, `template_day_routines`.
- **Sesiuni (istoric care nu se modifică):** `workout_sessions`, `session_exercises`, `session_exercise_muscle_groups`, `session_routines`, `workout_sets`.
- **Marketplace:** `template_votes`, `template_saves`, `template_use_events`.
- **Tabele pregătite pentru extinderi (nefolosite încă de cod):** `coach_session_comments`, `coach_template_assignments`, `template_analysis_results`, `outbox_events`.

Cele mai importante constrângeri pentru integritate sunt: regula că un program public trebuie să aibă o dată de publicare (`CHECK ((visibility='PUBLIC' AND published_at IS NOT NULL) OR visibility='PRIVATE')`); interdicția de a te antrena singur (`CHECK (coach_user_id <> client_user_id)`); indexul unic parțial pentru un singur antrenament activ per utilizator; indecșii unici parțiali de unicitate cu `WHERE deleted_at IS NULL`; și cheile externe care pot fi anulate (`ON DELETE SET NULL`) către datele sursă, puse în pereche cu coloanele de instantaneu. Definițiile-cheie sunt în Anexa 3.

Anexa 3 — Listări de cod reprezentative

Listarea 1. Felul determinist în care lucrează analizorul: fiecare sub-scor pornește de la maxim și scade o valoare fixă pentru fiecare problemă (extras simplificat din TemplateAnalyzerService).

```
private int scoreVolumeAndCoverage() {
    int score = 35;
    // O grupa minora nu poate masca o regiune majora absenta:
    if (!lowerBody) { add("NO_LOWER_BODY", HIGH, ...); score -= 12; }
    if (!pulling)    { add("NO_PULL",      HIGH, ...); score -= 12; }
    if (!pushing)    { add("NO_PUSH",     HIGH, ...); score -= 12; }
    for (String bucket : ALL_BUCKETS) {
        // serii saptamanale ponderate: PRIMARY = 1.0, SECONDARY = 0.5
        double volume = weeklyWeightedSets(bucket);
        if (volume <= 3) { add("MUSCLE_VOLUME_VERY_LOW", ...); score -= 3; }
        else if (volume > 25) { add("MUSCLE_VOLUME_EXCESSIVE", ...); score -= 5; }
        else if (volume > 20) { add("MUSCLE_VOLUME_HIGH", ...); score -= 2; }
        // ... alte benzi de volum ...
    }
    return Math.max(0, score);
}

private static String category(int score) {
    if (score >= 80) return "WELL_STRUCTURED";
    return score >= 55 ? "DECENT_STRUCTURE" : "NEEDS_REVIEW";
}
```

Listarea 2. Ordinea de autorizare (din SecurityConfig): RBAC pe rute, peste care coaching-ul adaugă verificarea ReBAC la nivel de serviciu.

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers(POST, "/api/auth/register", "/api/auth/login",
                        "/api/auth/refresh", "/api/auth/logout").permitAll()
    .requestMatchers(GET, "/api/health").permitAll()
    .requestMatchers("/api/admin/**").hasRole("ADMIN")
    .requestMatchers("/api/coach/**").hasRole("COACH")
    .requestMatchers("/api/**").authenticated())
```

```
.anyRequest().permitAll())
```

Listarea 3. Două definiții SQL importante: garda de concurență (V3) și tabela de outbox pregătită din V1 pentru extinderea cu outbox tranzacțional.

```
-- Un singur antrenament activ per utilizator (migrarea V3):
CREATE UNIQUE INDEX ux_workout_sessions_one_active_per_user
  ON workout_sessions (user_id) WHERE status = 'IN_PROGRESS';

-- Tabela de outbox, definita in V1, neutilizata inca de cod:
CREATE TABLE outbox_events (
  id          uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  aggregate_type varchar(100) NOT NULL,
  aggregate_id  uuid NOT NULL,
  event_type    varchar(100) NOT NULL,
  payload       jsonb NOT NULL DEFAULT '{}'::jsonb,
  status        varchar(30) NOT NULL DEFAULT 'PENDING',
  attempts      int NOT NULL DEFAULT 0,
  available_at  timestamptz NOT NULL DEFAULT now(),
  processed_at  timestamptz,
  created_at    timestamptz NOT NULL DEFAULT now(),
  last_error    text,
  CONSTRAINT check_outbox_status
    CHECK (status IN ('PENDING', 'PROCESSING', 'PROCESSED', 'FAILED'))
);
CREATE INDEX outbox_events_processing_idx
  ON outbox_events (status, available_at, created_at);
```

Anexa 4 — Rezultatele complete ale evaluării

Am măsurat la 7 dimensiuni ale setului de date (1k–2M), cu 300 de iterații măsurate (încălzire 30, buget de timp 15s; n arată câte eşantioane s-au strâns când bugetul a limitat o celulă). PostgreSQL: `tsvector`+GIN ordonat cu `ts_rank_cd`, fuzzy cu `pg_trgm`. OpenSearch: aceleași mapări și interogări ca în producție.

Corpus	Construcție PG	Construcție OS	Dimensiune PG	Dimensiune OS
1.000	216 ms	203 ms	1,6 MB	0,3 MB
10.000	1.926 ms	975 ms	14,6 MB	2,9 MB
50.000	7.966 ms	3.481 ms	68,7 MB	14,3 MB
100.000	14.006 ms	6.349 ms	135,5 MB	28,5 MB
500.000	68.752 ms	37.987 ms	664,7 MB	140,8 MB
1.000.000	125.774 ms	57.701 ms	1.313,6 MB	349,1 MB
2.000.000	239.192 ms	114.908 ms	2.632,5 MB	738,7 MB

Tabela 4.1: Timpul de construcție a indexului și dimensiunea pe disc, pe dimensiuni de corpus.

Corpus	PostgreSQL ($p_{50}/p_{95}/p_{99}$)	OpenSearch ($p_{50}/p_{95}/p_{99}$)
1.000	1,46 / 2,68 / 3,12	3,25 / 5,70 / 10,56
10.000	10,64 / 14,32 / 17,63	2,85 / 5,21 / 7,17
50.000	48,70 / 66,59 / 77,97	5,17 / 10,05 / 13,98
100.000	100,21 / 117,87 / 136,46	6,84 / 10,59 / 12,83
500.000	779,47 / 1021,25 / 1229,16	22,75 / 38,09 / 67,26
1.000.000	872,19 / 1199,43 / 1220,09	33,64 / 45,24 / 83,90
2.000.000	1863,97 / 2783,12 / 2783,12	63,65 / 73,05 / 111,67

Tabela 4.2: Latența căutării full-text „bench press” (ms), pe dimensiuni de corpus.

Corpus	PostgreSQL ($p_{50}/p_{95}/p_{99}$)	OpenSearch ($p_{50}/p_{95}/p_{99}$)
1.000	14,45 / 18,91 / 21,69	2,37 / 3,74 / 5,63
10.000	127,93 / 134,06 / 136,22	3,23 / 7,38 / 10,94
50.000	621,74 / 645,63 / 646,08	2,66 / 4,95 / 6,59
100.000	1831,47 / 2169,61 / 2169,61	3,15 / 5,74 / 7,38
500.000	2447,22 / 8354,43 / 8354,43	4,06 / 7,20 / 8,95
1.000.000	7806,25 / 17790,10 / 17790,10	3,94 / 9,18 / 13,35
2.000.000	15825,49 / 16399,71 / 16399,71	3,89 / 9,38 / 14,14

Tabela 4.3: Latența căutării fuzzy „benhc” (greșeală de tastare, ms): cum se degradează `pg_trgm` față de OpenSearch, care rămâne aproape constant.

Anexa 5 — Contractul API (selecție)

Toate endpoint-urile, în afară de cele de autentificare și de starea de sănătate, cer un token JWT valid. Erorile folosesc un format uniform, cu un cod stabil (de exemplu TEMPLATE_NOT_FOUND, ACTIVE_WORKOUT_EXISTS, SEARCH_UNAVAILABLE).

Autentificare	: POST /api/auth/{register,login,refresh,logout}; GET /api/auth/me
Planificare	: /api/exercises; /api/routines; /api/gyms; /api/gyms/{id}/equipment; /api/equipment/{id}
Programe	: /api/templates; /api/templates/{id}; POST /api/templates/{id}/{publish,unpublish}
Sesiuni	: POST /api/workouts/start; GET /api/workouts/active; POST /api/workouts/{id}/{finish,cancel}; POST /api/session-exercises/{id}/sets; PUT DELETE /api/sets/{id}
Istoric	: GET /api/history; GET /api/analytics/overview
Marketplace	: GET /api/marketplace/templates; POST /api/marketplace/templates/{id}/{vote,save,use}
Analizor	: GET /api/templates/{id}/analysis
Cutare	: GET /api/search/templates; GET /api/search/workouts; POST /api/admin/search/reindex (doar ADMIN)
Coaching	: /api/coach/clients/... (rol COACH + relatie activa); /api/coaching/{invites,coaches}

Anexa 6 — Configurație și rulare

Tot stack-ul rulează prin Docker Compose (PostgreSQL 16, OpenSearch 2.13 cu un volum persistent și aplicația). Indicatorul `app.search.enabled` pornește sau oprește căutarea.

```
# application.yml (extras)
spring:
  jpa:
    hibernate:
      ddl-auto: validate          # schema detinuta de Flyway
  app:
    security:
      jwt:
        issuer: workout-thesis
        access-ttl: 15m
        refresh-ttl: 14d
    search:
      enabled: ${APP_SEARCH_ENABLED:false}
      uri: ${APP_SEARCH_URI:http://localhost:9200}
      templates-index: templates_v1
      sessions-index:  workout_sessions_v1
```

Rularea cu date demonstrative (istoric din 2020 până azi) se face cu profilul `demo`, care la pornire reconstruiește indexul de căutare:

```
SPRING_PROFILES_ACTIVE=demo docker compose up --build
```